

Supervised Learning Report FAQ

CS7641: Machine Learning

Spring 2026

1 Frequently Asked Questions (FAQ)

The assignment is open-ended in analysis, but expectations are concrete. This FAQ is designed to prevent common mistakes and reduce rework.

A. Scope and minimum requirements

Q: Exactly what must I run?

Required: You must run **four algorithm families** on **both** datasets:

1. Decision Trees (DT) with pruning/regularization,
2. k-Nearest Neighbors (kNN),
3. Support Vector Machines (SVM) with ≥ 2 kernels, and
4. Neural Networks (NN) using **SGD only**.

Neural network requirement (important): For **each dataset**, you must train:

1. one NN using **scikit-learn** (`MLPClassifier` with SGD), and
2. one NN using **PyTorch** (a compact MLP trained with SGD).

Best practice: Keep preprocessing as consistent as possible across models so comparisons are fair.

Q: What are the datasets and tasks for Spring 2026?

Required: You must use both datasets provided on Canvas.

- **Dataset A (Adult Income):** Binary classification. Target: `income` ($\leq 50K$ vs $> 50K$).
- **Dataset B (Wine Quality):** Multiclass classification. Target: `quality` (discrete rating/class).

Best practice: Report class counts for both datasets early in the report. It helps contextualize your metrics.

Q: What metrics are required?

Required:

- **Adult (binary classification):** Accuracy and F1 (minimum). If you report threshold-based results, include **PR-AUC**.
- **Wine (multiclass classification):** Accuracy and **Macro-F1** (minimum). Include a confusion matrix and per-class discussion.

Best practice:

- For Adult, report the **class prevalence** and interpret PR-AUC relative to that baseline.
- For Wine, include per-class precision/recall or a short “which classes are hardest” analysis.

Q: What figures and tables are required for each algorithm?

Required for each algorithm on each dataset:

- **Learning curve (LC):** training and validation metric vs. training size.
- **Model-complexity curve (MC):** validation metric vs. one hyperparameter.
- **Runtime table:** fit time and predict time (wall-clock) with a short hardware note.
- **Confusion matrix:** for the final selected model configuration (per dataset, per algorithm).

Best practice: If space allows, include the training curve on the MC plot (not just validation) to make bias/variance trends clearer.

Q: What does each graph actually need to show? (More detail)

Your plots are not decoration. Each one should support an argument about bias/variance, sensitivity, or efficiency.

Learning Curve (LC). **Required:** A learning curve shows how performance changes as you give the model more data.

- **X-axis:** number of training samples (use 4–8 points).
- **Y-axis:** your primary metric (Adult: F1 or PR-AUC; Wine: Macro-F1 or accuracy).
- **Lines:** training score and validation score.

How to interpret:

- If both curves are low and close together, the model is likely **high bias** (underfitting).
- If training is high but validation is much lower, the model is likely **high variance** (overfitting).
- If the validation curve is still rising sharply at your largest training size, you are likely **data-limited**.

Model-Complexity Curve (MC). **Required:** An MC curve isolates one hyperparameter and shows how it changes performance.

- **X-axis:** one hyperparameter with ≥ 4 values.
- **Y-axis:** validation metric (and training metric if possible).
- **Hold everything else fixed.**

Examples of valid MC “knobs”:

- DT: `max_depth` or `ccp_alpha`
- kNN: k
- SVM: C (and optionally γ for RBF, but do one at a time for interpretability)
- NN: width, L2 regularization, or learning rate (one at a time)

How to interpret:

- You should be able to describe why too-small vs. too-large values increase bias or variance.
- If performance is flat across the range, say so and explain what that suggests about model sensitivity.

Epoch Curve (NN only). **Required for both sklearn and PyTorch NNs:** loss/metric vs. epoch to show optimization dynamics.

- **X-axis:** epoch number.
- **Y-axis:** loss and/or metric (train and validation).
- Mark the **early-stopping** epoch if you use it.

How to interpret:

- Diverging loss: learning rate too high, poor initialization, or unstable batch dynamics.
- Training improves but validation worsens: classic overfitting, regularize or reduce capacity.
- Both curves stagnate: underfitting or optimization stuck (adjust LR schedule, width, or regularization).

Confusion Matrix (classification). **Required:** Present a confusion matrix for the final configuration.

- For **Adult**: interpret FP vs. FN and relate to the threshold choice if you changed it.
- For **Wine**: call out which classes get confused (e.g., 5 vs. 6) and why that might happen.

Runtime Table. **Required:** Show fit time and predict time for the final model per algorithm (and dataset).

- Report **wall-clock times** (seconds or minutes).
- Include a one-line **hardware note** (CPU type, RAM, GPU if used for NN).

How to interpret:

- kNN prediction cost often dominates.
- Kernel SVM may scale poorly in n .
- NNs can be stable in runtime but sensitive to training budget (epochs, batch size).

Q: What is the “Leakage / Evaluation Debug” rule?

Required: If your results look suspiciously good (e.g., implausibly high accuracy or near-perfect Macro-F1), you must include a **Leakage / Evaluation Debug** subsection that argues your results are real.

Acceptable diagnostics include:

- a shuffled-label baseline (performance should collapse),
- a dummy baseline (e.g., predict most frequent class),
- verifying that preprocessing is fit on training only (no peeking), or
- checking that your “test” set did not leak into tuning decisions.

Best practice: Include at least one baseline number in your Results table so your gains are interpretable.

B. Evaluation setup and correctness

Q: How should I split the data?

Required: Split once into **train/test**. Perform all hyperparameter tuning using only the **training set** (e.g., cross-validation within train). Evaluate the final selected model **once** on the held-out test set.

Required: Fix random seeds and report them so your splits and results are reproducible.

Best practice: Use **stratified** splits for classification to preserve class proportions.

Q: Do I need cross-validation?

Required: Yes. For hyperparameter tuning, you must use either **cross-validation** (recommended) **or** an explicit **train/validation split**. You may not tune on the test set.

What cross-validation means (in this assignment): Cross-validation (CV) estimates validation performance using multiple splits of the **training data**. Instead of relying on a single validation split, CV trains and evaluates several times and aggregates results. This helps reduce sensitivity to a “lucky” or “unlucky” split.

A standard workflow to follow:

1. Split the dataset once into **train** and **test**. Keep the test set held out for final evaluation.
2. Use **only the training set** for tuning:
 - either using k -fold cross-validation (recommended), or
 - using a single train/validation split (allowed if runtime is restrictive).
3. Choose hyperparameters using validation results.
4. Retrain a final model using the chosen hyperparameters on the training data.
5. Evaluate on the held-out test set and report that as final performance.

Best practice (recommended CV settings):

- Use **Stratified k -fold CV** for classification (to preserve class proportions in each fold).
- Typical values: $k = 5$ folds (balanced trade-off) or $k = 3$ folds (faster).
- For reproducibility: fix `random_state` and shuffle when appropriate.
- If the dataset has rare classes, verify that each fold contains at least a few examples of each class.

Common mistakes (avoid these):

- **Using the test set as validation during tuning.** This inflates reported performance and weakens the analysis.
- **Reporting the best single fold score.** CV is intended to summarize across folds; report the mean and consider reporting variability.
- **Preprocessing leakage inside CV.** If you scale/encode/impute, fit these transformations on training folds only (pipelines help ensure this happens correctly).

What to include in your Methods section:

- Split strategy (train/test ratio and whether it is stratified).
- Tuning strategy (CV with $k = ?$ folds or a single train/validation split).
- Random seed(s) used.
- The metric used to select hyperparameters (Adult: F1 or PR-AUC; Wine: Macro-F1).

If runtime makes CV difficult: A single train/validation split is acceptable. If you choose this option, briefly justify the decision with runtime evidence and keep the held-out test set reserved for the final evaluation.

Best practice: Use stratified k -fold cross-validation (e.g., $k = 5$) unless runtime is prohibitive.

Q: Can I use only a subset of the dataset to reduce runtime?

Allowed (with justification): Yes. If runtime is prohibitive, you may train and tune on a subset that is approximately **one order of magnitude smaller** than the full dataset (e.g., $N \rightarrow N/10$).

Required: You must justify subsampling with **runtime evidence**. At minimum, include:

- the original dataset size and the subset size used,
- wall-clock fit/predict time for at least one larger trial run (or a failed/terminated run with timestamps),
- wall-clock fit/predict time for the chosen subset, and
- a brief explanation of the trade-off (runtime vs. performance).

Required: Your subset must preserve label distribution (use stratified sampling).

Q: Do I need to scale features (e.g., StandardScaler)?

Required: Yes, when justified. Feature scaling is required for **SVM and kNN** and is strongly recommended for **NNs**. Decision Trees do not require scaling.

Justification requirement: If you apply scaling, you must include a short justification grounded in EDA. For example, cite feature ranges, skew, or mixed-units numeric inputs that would distort distance/margins.

Best practice:

- Apply scaling **only to numeric features**.
- Fit scalers on **training folds only**, then apply to validation/test.
- Leave already-binary/one-hot encoded features unchanged.

Q: How do I choose a threshold for Adult (binary classification)?

Required: If you report a confusion matrix at a non-default threshold, justify the rule (e.g., maximize F1 on validation).

Q: What should I do if Wine has rare classes?

Required: You must still treat Wine as multiclass classification using the quality ratings as classes.

Best practice:

- Use stratified splitting.
- Report Macro-F1.
- Discuss which classes are hardest and why.

C. Algorithm-specific questions (what we expect and common pitfalls)

Q: What does “Decision Trees with pruning” mean?

Required: You must apply at least one complexity control mechanism and show a model-complexity curve:

- `max_depth`, `min_samples_leaf` / `min_samples_split`, or `ccp_alpha`.

Q: What kernels are required for SVM?

Required: Use ≥ 2 kernels per dataset:

- **Linear** SVM and **RBF** SVM.

Q: How many k values should I test for kNN?

Required: Test meaningfully different values:

$$k \in \{3, 5, 11, 21\}.$$

Q: What exactly is required for the Neural Networks?

Required: For each dataset, train and report:

1. **scikit-learn NN:** `MLPClassifier` using **SGD only**.
2. **PyTorch NN:** a compact MLP trained using **SGD only**.

Q: What does “SGD only” mean in practice?

Required: You may **not** use momentum, Nesterov, Adam/AdamW, RMSprop, Adagrad, or other adaptive optimizers.

Q: Is Grid Search required for hyperparameter tuning?

Required: No. Grid Search is not required and is often not a good use of computation.

Recommended alternatives (best practices):

- Random search over sensible ranges.
- Coarse-to-fine search: broad sweep, then zoom into the promising region.
- Log-scaled exploration for C , γ , learning rate, and regularization.
- Bayesian optimization (e.g., Optuna) for efficient trial allocation.
- Literature-informed priors and search ranges.

Reporting requirement (separate from search method): Regardless of how you tune, you must include a **model-complexity curve** for each algorithm that varies **one** hyperparameter across at least **4 values** while holding other settings fixed. This curve is for interpretation, not necessarily your entire tuning strategy.

D. EDA, hypotheses, and interpretation

Q: What must be included in the EDA?

Required:

- dataset shape and feature types,
- missingness,
- class distribution,
- summary statistics for numeric features,
- at least one visualization per dataset.

Q: What is a good hypothesis for this assignment?

Required: After EDA, state at least one hypothesis per dataset *before* tuned results.

E. Reproducibility, submission, and style

Q: Do I need a README.md in my repo?

No. A separate README is not required.

Q: What must be included in the reproducibility PDF (REPRO)?

Required:

- read-only Overleaf link,
- a single Git commit hash,
- exact run commands and environment details,
- dataset access instructions,
- EDA summaries for both Adult and Wine.

Q: Do I need outside sources?

Required: Yes, at least **two peer-reviewed references** beyond course materials.

Q: Can I use AI tools (LLMs)?

Allowed with disclosure. You must disclose tool usage and may not submit AI-written analysis as your own.

Q: How do I fit all required figures within the page limits?

This is a design problem, not a suffering problem. You should plan your figures so each one carries multiple requirements.

Best practice strategies that graders love:

- **Use multi-panel combined figures.** For example, make one figure per dataset that contains:
 - a 2×2 grid of learning curves (DT, kNN, SVM, NN), and
 - a second 2×2 grid of model-complexity curves.
- **Make one shared runtime table per dataset.** One compact table can cover all four algorithms.
- **Compress confusion matrices.** You can place 4 confusion matrices for a dataset into one 2×2 combined image.
- **Prefer “summary + one diagnostic” over 10 micro-figures.** A single well-chosen learning curve plus one MC curve often tells the story better than many plots.
- **Use consistent axes and formatting.** If all learning curves share the same x-axis scale and metric, they are easier to compare and take less space.

What not to do:

- Do not include repeated near-identical plots for tiny hyperparameter changes.
- Do not include 4 separate runtime tables (one per algorithm) if a single table suffices.
- Do not spend half a page restating what the legend already says.

F. Quick self-check (use this before submitting)

- **Tasks:** Adult binary classification; Wine multiclass classification.
- **Algorithms:** DT, kNN, SVM (2+ kernels), NN (sklearn + PyTorch).
- **NN rule:** SGD only, no momentum/Adam/RMSprop.
- **Scaling:** used for SVM/kNN and justified (recommended for NN).
- **Tuning:** search strategy is flexible; include a clean MC curve (≥ 4 points) per algorithm.
- **Plots:** LC + MC + runtime + confusion matrices (combined figures encouraged).
- **EDA:** documented and used to motivate hypotheses.
- **Splits:** tune on train only; test once at the end; seeds fixed.
- **Sanity:** Leakage/Evaluation Debug included if results are “too perfect.”
- **Citations:** two peer-reviewed references, consistent style.
- **Repro:** Overleaf link + commit hash + commands + seeds + EDA summary in REPRO PDF.